Bike Rental Write-Up

This Bike Rental project is my first true solo data engineering project. So, I kept the central goal of the entire project in mind “A bike rental company has asked you to create a database to help their analysts understand the effects of weather on bike rentals. “. The data provided for this project is a 2016 (full-year) Newark, New Jersey, U.S.A weather data CSV, and 12 months of bike rental CSVs (1 file per month) from Citi Bike. In Python, I cleaned and transformed the data appropriately. I dropped two columns in the weather dataframe (PGTM & TSUN) because they were null for all rows and seemed unnecessary. I also dropped some nulls in the bike rental dataframe where column is “User Type”, because it would have minorly skewed our analysis having nulls in summary statistics, and it was only 380 rows gone whilst having 247,204 rows remaining. Connecting to the SQL database (pgAdmin4) was fairly successful, but because I have never done it in Python before I had to ask some help from ChatGPT (I commented that in my Jupyter Notebook). I had to rename columns in all lower-case, so that no errors occurred when sending data to the SQL database.

The first database schema I created kept the weather data in a complete table. Then, I decided for the next two tables to be the split bike rental data into a one-to-one relationship. The 1st table was called main_bike_data, and had columns: main_id (primary key), trip_duration, bike_id, user_type, birth_year, and gender. The 2nd table was named bike_trip_data and had columns: trip_id (primary_key), main_id (foreign_key), start_time, stop_time, start_station_id, start_station_name, start_station_latitude, start_station_longitude, end_station_id, end_station_name, end_station_latitude, and end_station_longitude. Though admittedly, I ran into problems with this original table schema. I had a fundamental mis-conception that PostgreSQL would create foreign key numbers for me whilst not understanding I had to generate

them myself or already have a dedicated column present in the data. So, after understanding more, I chose to keep all of the bike rental data together into one relational table. The new schema had a one-to-many relationship from the weather data table to the bike data table.

In conclusion to the project, I created three SQL data views after having both tables set up properly in PgAdmin4. The data views answered questions: How many bike riders are there a day accounting for the collective trip durations as well? What amount of bike riders are 'Subscribers' rather than 'Customers'? And lastly, how do weather conditions (temperature average, precipitation, and wind speed averages) affect total trips a day? These questions are simply general prompts that may be useful for the analyst teams. *To see SQL data views, please see the Github link to the SQL code.